Assignment Instructions

Listen

DOM Manipulation Assignment — Interactive Study Task List

Included files:

styles.css

index.html

Course: Introduction to JavaScript **Topic:** Working with the DOM **Points:** 100 (+ up to 20 bonus)
Difficulty: Beginner — built so you can succeed using the class demo as your guide.

1. What you are building

You are given a finished web page: the HTML and CSS are done. The only thing missing is the JavaScript — and that's your job.
When you are finished, your page will:

1. **Count** the tasks in the list and keep the number correct
2. **Add** a task the user types into the box
3. **Remove** the last task (without crashing when the list is empty)
4. **Show** a random motivational message
5. **Mark** all tasks as done / not done with one button

Everything you need is a pattern you have already seen in the class demo (dom-basics-demo). This assignment is the *same skills* on a new page.

2. What is provided vs. what you write

File	Who writes it	What it does

index.html	Provided — **do not edit**	The page structure and all element IDs
styles.css	Provided — **do not edit**	The look of the page, including the .done style
script.js	**You**	Contains 6 TODO markers to complete

Element map (everything your JS targets)

ID	Element	Used for
message	paragraph	All feedback text ("Added: ...", errors, quotes)
task-input	input box	Where the user types a task
task-list		The list your JS adds/removes items in
task-count	paragraph	"Total Tasks: N"
btn-add, btn-remove, btn-motivate, btn-done-all	buttons	The four buttons you wire up

3. Setup

1. Copy the assignment folder (keep the three files together).
2. Open index.html in your browser (double-click it).
3. Open DevTools (**F12**) → **Console** tab and keep it open while you work.
4. Open script.js in your editor and search for **TODO**.

Work in small steps: **edit** → **save** → **refresh** → **test** → **check the Console.**

4. Your six steps

Each step tells you *where* to work, *what* to do, and gives you a pattern from the demo. The TODO numbers in script.js match these steps.

Step 1 — Wire up the buttons (10 pts)

Goal: Make three buttons actually do something.

In setupEventListeners(), one button is already connected for you:

```
btnAdd.addEventListener('click', addTask);
```

Connect the other three the same way:

Variable	Should call this function
btnRemove	removeLastTask
btnMotivate	changeMessage
btnDoneAll	markAllDone

⚠ **Most common mistake in the whole course:** writing addTask() with parentheses inside addEventListener. Parentheses run the function *immediately*. Write the function **name only** so it runs when clicked.

Test: Click each button. Nothing should crash (most won't do much yet — that's expected until the next steps).

Step 2 — Keep the counter updated (10 pts)

Goal: #task-count always shows the true number of tasks.

In updateCount():

```
const total = taskList.querySelectorAll('.task-item').length;
taskCount.textContent = `Total Tasks: ${total}`;
```

The init() function (already written) calls updateCount() when the page loads — refresh and confirm the counter says **Total Tasks: 3**.

Step 3 — Add a task (25 pts)

Goal: Typing a task and clicking *Add Task* puts it on the list.

In addTask(), complete the five small parts (3a–3e) **in order**:

1. **Read** what was typed: taskInput.value.trim()
2. **Guard:** if the text is empty, show a message and return; (an empty <li> looks broken and loses points)
3. **Create:** document.createElement('li'), then set li.className = 'task-item' and li.textContent = text
4. **Insert:** taskList.appendChild(li)
5. **Finish:** clear taskInput.value, call updateCount(), show a success message

Test: Type "Buy groceries" → Add. It should appear, the counter should rise to 4, and the box should empty. Then click Add with an empty box — you should see your friendly error message instead of a blank task.

Step 4 — Remove the last task (15 pts)

Goal: *Remove Last Task* removes the bottom item — and never crashes.

In removeLastTask():

1. Select all tasks: taskList.querySelectorAll('.task-item')
2. **Guard:** if items.length === 0, show a message and return;

3. Save the last item (and its text) in a variable
4. Remove it: taskList.removeChild(lastItem) **or** lastItem.remove()
5. Call updateCount() and confirm what was removed

Test: Remove until the list is empty, then click again — you must see your message, not a red error.

Step 5 — Motivate Me! (15 pts)

Goal: Each click shows a different random quote in #message.
In changeMessage():

```
const index = Math.floor(Math.random() * QUOTES.length);
message.textContent = QUOTES[index];
```

QUOTES is already provided at the top of the file. This is exactly how “Change Name” worked in the class demo.

Step 6 — Toggle all tasks (15 points)

Goal: One button marks every task done (click again to un-mark).
In markAllDone():

```
const items = document.querySelectorAll('.task-item');
items.forEach(function (item) {
    item.classList.add('done');   // or classList.toggle('done')
});
```

The CSS already styles .done (strikethrough + green) — you only add the class. Using toggle makes the button work in both directions.

5. Rules (these are graded)

1. Use **textContent**, never innerHTML, for anything the user typed.
2. Never allow an **empty task** into the list (Step 3 guard).
3. Never crash when the list is **empty** (Step 4 guard).
4. Do **not** edit index.html or styles.css.
5. Your own work — the class demo and cheat sheet are allowed references.

6. Self-check before you submit

- All four buttons do something when clicked
- Counter is correct after adding and after removing
- Adding with an empty box shows a message, adds nothing
- Removing from an empty list shows a message, no red error
- Motivate Me! shows different quotes on different clicks
- Toggle All styles every task; clicking again un-styles them
- Refreshing the page returns everything to the starting state
- Console shows **no red errors** while you test every button

7. Bonus (up to +20)

Bonus	Points	Hint
Click a single task to mark it done	+10	Add item.addEventListener('click', ...) with classList.toggle('done') inside addTask(). Partial credit if only new tasks work; **full credit** if the 3 starter tasks also respond (hint: the demo uses *event delegation* on the list, or loop over existing items once).
Press Enter in the input box to add a task	+10	See the keyboard example in the class demo: listen for 'keydown' and check event.key === 'Enter'.

8. Rubric

Requirement	Points	Partial credit
Step 1 — three buttons wired correctly	10	3 pts per button
Step 2 — counter updates correctly	10	5 if it works only after add
Step 3 — add task (read, guard, create, insert, finish)	25	graded part by part (3a–3e)
Step 4 — remove last (guard included)	15	8 without the empty-list guard
Step 5 — random motivational message	15	8 if it always shows the same quote
Step 6 — toggle all tasks done	15	8 if tasks are styled but toggle doesn't undo

Follows the rules, readable code, some comments	10	—
Total	**100**	
Bonus: click a task	+10	
Bonus: Enter key adds a task	+10	

9. Submission

1. Save all three files.
2. Test one final time with the Console open (no red errors).
3. Zip the folder: LastName_FirstName_Assignment.zip (must contain index.html, styles.css, script.js).
4. Submit via the course submission page before the deadline.

10. Troubleshooting

Symptom	Likely cause	Fix
Clicking a button does nothing	Button not wired, or typo in the function name	Re-check Step 1 spellings
Nothing works / red error on load	Syntax error (missing } or)	Console shows the line number; count your braces
Uncaught ReferenceError: xyz is not defined	Misspelled variable/function, or using something before defining it	Match names exactly with the element map
Function runs as soon as the page loads	You wrote functionName() inside addEventListener	Remove the ( )
Counter shows undefined	Missing .length or wrong variable	Re-check Step 2
New task looks unstyled (plain bullet)	Forgot li.className = 'task-item'	Step 3c
Blank task appears	No empty-input guard	Step 3b
Red error when list is empty	No empty-list guard	Step 4b

Two tasks added per click	The same button got wired twice	Look for a duplicate addEventListener line

---

Submission

Click to add text

Word count: 0

Drag files here

Maximum size: 512 MB

or

Upload a file **Upload from cloud**

Save and Close **Submit**